FPGA RTL Experiment

Mixed-Precision Multiplier–Accumulator (MAC) — Verilog-2001

Design a signed multiplier with different operand widths and a wider accumulator.

Includes synthesizable RTL, a testbench, worked examples, and a simulation-to-FPGA workflow.

RTL → simulation → synthesis → hardware validation

1. Experiment objectives

Multiply two signed operands that have different bit widths.

Preserve the full product width: PROD_W = A_W + B_W.

Sign-extend the product into a wider accumulator.

Accumulate one product per enabled clock edge.

Support synchronous reset, accumulator clear and enable control.

2. MAC operation

acc_next = acc + (a × b)

On each rising clock edge:

- rst = 1 → acc becomes zero
- otherwise clear = 1 → acc becomes zero
- otherwise en = 1 → acc adds the current signed product
- otherwise → acc holds its previous value

This is a sequential datapath: multiplication is combinational; accumulation is registered.

3. Operand and accumulator widths

Example parameters

A_W = 8 bits

B_W = 12 bits

ACC_W = 24 bits

Product width:

PROD_W = 8 + 12 = 20 bits

Why use a wider accumulator?

Repeated additions need more range than one product.

The 20-bit signed product is sign-extended to 24 bits before addition, preserving its signed numeric value.

Constraint: ACC_W must be greater than or equal to A_W + B_W in this implementation.

4. Datapath diagram



The accumulator output feeds back to the adder to form the next accumulated result.

5. RTL module interface

Inputs

- clk — rising-edge clock
- rst — synchronous active-high reset
- clear — clear accumulator
- en — enable accumulation
- a, b — signed operands

Output

- acc — signed ACC_W-bit sum

Parameters

- A_W: width of operand a
- B_W: width of operand b
- ACC_W: accumulator width

The RTL uses Verilog-2001 syntax: parameter, wire, reg, assign and always @(posedge clk).

6. RTL: product and sign extension

```
                    // Verilog-2001: Mixed-precision signed multiply-accumulate (MAC)
// ACC_W must be >= A_W + B_W for a full-precision product to fit.
module mixed_precision_mac #(
    parameter A_W   = 8,
    parameter B_W   = 12,
    parameter ACC_W = 24
) (
    input  wire                     clk,
    input  wire                     rst,       // synchronous active-high reset
    input  wire                     clear,     // clear accumulator
    input  wire                     en,        // accumulate when high
    input  wire signed [A_W-1:0]    a,
    input  wire signed [B_W-1:0]    b,
    output reg  signed [ACC_W-1:0]  acc
);

    localparam PROD_W = A_W + B_W;

    wire signed [PROD_W-1:0] product;
    wire signed [ACC_W-1:0]  product_ext;

    // Full-precision signed product.
    assign product = a * b;

    // Sign-extend product to accumulator width.
    assign product_ext = {{(ACC_W-PROD_W){product[PROD_W-1]}}, product};

    // Sequential accumulator.
    always @(posedge clk) begin
```

7. RTL: sequential accumulator

```
                                                    if (rst)
            acc <= {ACC_W{1'b0}};
        else if (clear)
            acc <= {ACC_W{1'b0}};
        else if (en)
            acc <= acc + product_ext;
    end

endmodule
```

8. Worked simulation examples

After reset, the accumulator starts at 0.

1. a = 3, b = 5 → product = 15 → acc = 15
2. a = −2, b = 7 → product = −14 → acc = 1
3. a = 10, b = 20 → product = 200 → acc = 201
4. en = 0 → accumulator holds at 201
5. clear = 1 → accumulator becomes 0

The provided testbench prints actual results for comparison.

9. Testbench and verification

Compile mixed_precision_mac.v and tb_mixed_precision_mac.v in a Verilog-2001 simulator.

Set tb_mixed_precision_mac as the simulation top.

Check reset, positive and negative multiplication, accumulation enable and clear.

Inspect waveforms for clk, rst, clear, en, a, b and acc.

Add boundary-value tests and long accumulation sequences before hardware deployment.

10. FPGA implementation workflow

Simulate and verify the arithmetic first.

Synthesize the MAC and inspect multiplier/adder resource usage and timing.

Check that the selected FPGA supports the operand widths and desired clock frequency.

Apply clock and I/O constraints; implement and program the board.

Compare hardware accumulator values against the reference testbench.

Consider pipelining if the multiplier-plus-adder path does not meet timing.

11. Design considerations

Choose ACC_W based on the maximum number and magnitude of accumulated products.

The accumulator wraps on signed overflow in this basic design; saturation/overflow detection can be added if required.

Reset and clear are synchronous and active-high.

Inputs are sampled by the combinational multiplier; accumulation occurs at the next rising edge when en is high.

ACC_W must be at least A_W + B_W because the product is sign-extended using a parameterized replication.

Experiment deliverables

1. mixed_precision_mac.v — synthesizable Verilog RTL
2. tb_mixed_precision_mac.v — Verilog testbench
3. This presentation — datapath, width calculations, verification and FPGA workflow

Next: simulate → inspect waveforms → synthesize → validate on FPGA.